Statistical Machine Learning Final Project: Predictions of Flight Delays at the Pittsburgh Airport

Ellie Huang (elliehua) Nikita Jayaprakash (njayapra)

Executive summary

1 Introduction

The goal of this analysis is to predict whether or not a departing flight from Pittsburgh International Airport (airport code PIT) will be delayed based only on information available before departure. The dataset is flight information from 2023 - 2025 extracted from the Airline On-Time Performance Data from the Bureau of Transportation Statistics of the U.S. Department of Transportation. In addition to metrics about flight delays, it also reflects commercial flight activity to and from PIT. Based on the exploratory data analysis, feature engineering was used to develop additional predictors. This analysis compares several machine learning methods for this binary classification task, including logistic regression (with and without LASSO regularization), XGBoost, random forest (with and without rebalancing), and ensemble methods based on both simple and weighted averages of predicted probabilities. The metrics of model performance are training set misclassification rate, testing set misclassification rate, and area-under-the-curve (AUC). Based on the findings, the weighted ensemble achieved the strongest visible-set AUC of 0.6947, narrowly outperforming the simple average ensemble (0.6914), logistic regression (0.6902), and tuned XGBoost (0.6897). Overall, these results suggest that there is useful signal in the available pre-departure information, though performance remains limited, which may reflect both the difficulty of the problem and information missing from the feature set.

2 Exploration

2.1 Exploratory Data Analysis

Due to the data imbalance in Table 1, where there are more on-time flights than delayed, accuracy alone is not sufficient for comparing model performance, so this analysis places greater emphasis on AUC.

From Figure 1, delay rates vary meaningfully across carriers, so carrier should be a useful predictor. At the same time, some carriers have much lower flight counts at PIT than others, so raw group means will be noisy. That motivates using smoothed historical delay rates later in feature engineering instead of unsmoothed averages.

Table 1: The departure delay distribution is very imbalanced with some missing values, which may indicate cancelled flights.

	Flight Outcome	Count
	On Time	59534
	Delayed	10972
	NA	950
Total	—	71456

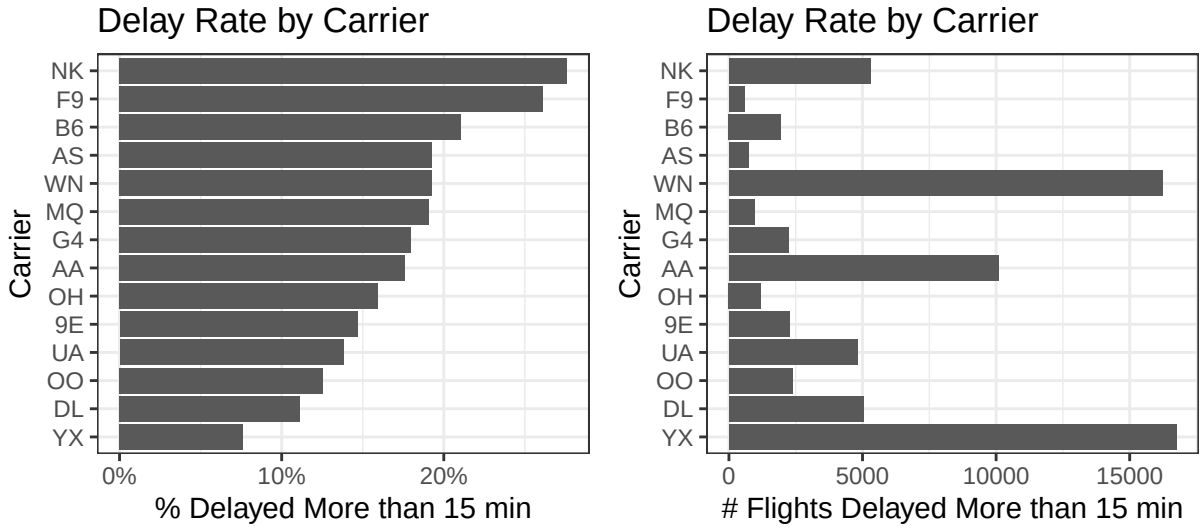

Figure 1: Delay rates and delay counts vary across different carriers, but the carriers with the highest delay rates don't necessarily have the highest delay counts.

Figure 2 shows that early flights start the day with fewer disruptions from previous legs, while later flights are more exposed to effects from earlier problems. This makes scheduled departure hour one of the strongest single predictors in the dataset.

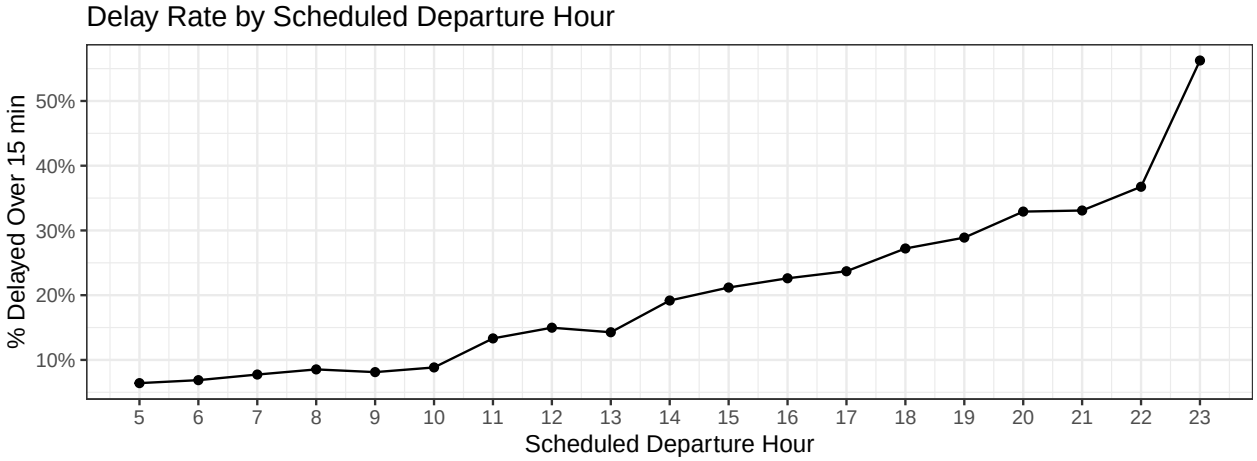

Figure 2: Delay rates rise steadily over the course of the day and are highest in the evening

Table 2: A small number of days have unusually high delay rates and delay counts.

Flight Date	Delay Rate	Count
January 11	0.3750000	176
August 04	0.2894737	190
August 06	0.2819149	188
August 05	0.2746114	193
March 10	0.2700000	200
July 12	0.2440191	209
August 02	0.2401961	204
May 10	0.2390244	205
May 05	0.2367150	207
July 01	0.2361809	199

Figure 3 shows that delays are worse in the summer months and in December, which is consistent with heavier travel volume and more weather-related disruption. Since these seasonal peaks are not spread evenly across the month, this supports adding a simple calendar feature rather than relying only on month.

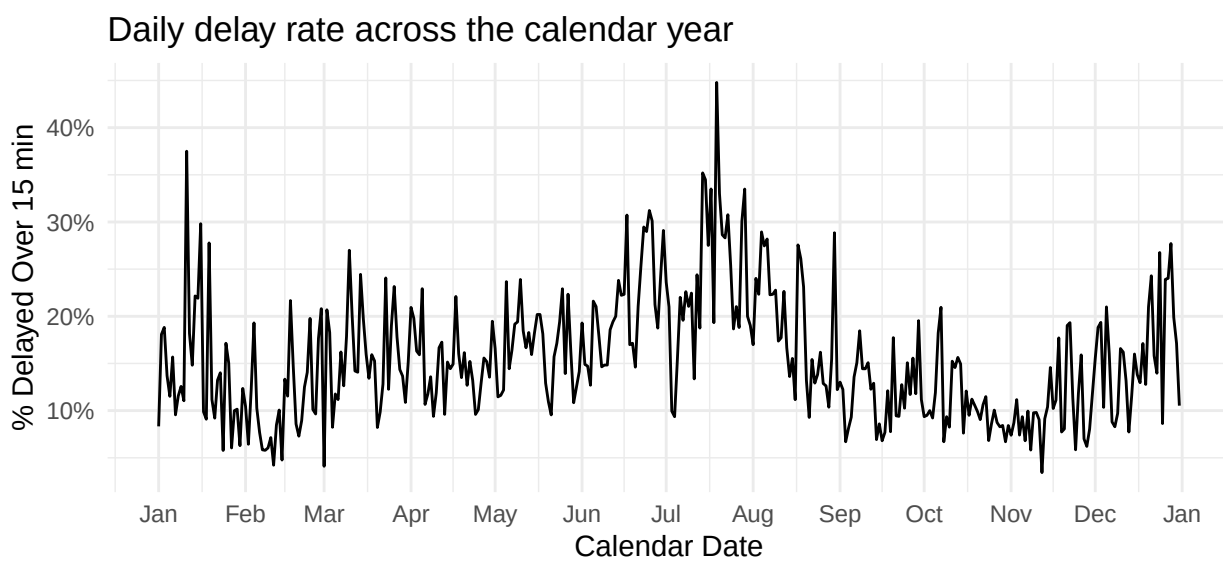

Figure 3: There are seasonal peaks, but the delays are not spread evenly across the month.

Further investigating specific calendar days with high delay rates, Table 2 suggests that there are broader operating shocks affecting many flights at once. That motivates adding dynamic same-day features based on earlier visible flights, along with a historical calendar-day disruption feature learned from the training data.

Figure 4 shows that delay rates differ across destinations, even among the busiest PIT routes, which suggests that route context adds information beyond carrier and time of day alone. Longer routes also show higher delay rates on average, so distance should remain in the model and may be easier to capture through both a raw continuous version and a bucketed factor version.

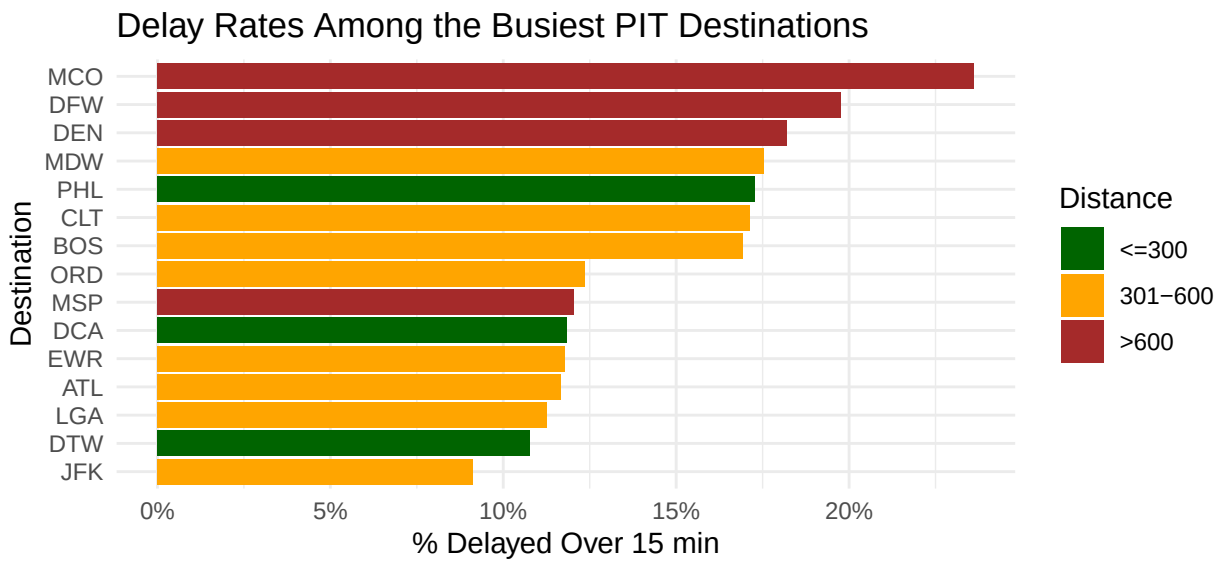

Figure 4: Longer routes and the specific destinations tend to have higher delay rates.

2.2 Dimension Reduction

This analysis applied principal component analysis (PCA) to a subset of variables that appeared potentially relevant from the EDA, including month, day of the month, day of the week, carrier, distance, and destination. As shown in Figure 5, while the first two principal components create visible clusters, they do not appear to separate delayed and on-time flights in a useful way. Because the class separation is weak, the principal components were not used for feature engineering, and the feature engineering instead followed the structure suggested by the exploratory data analysis.

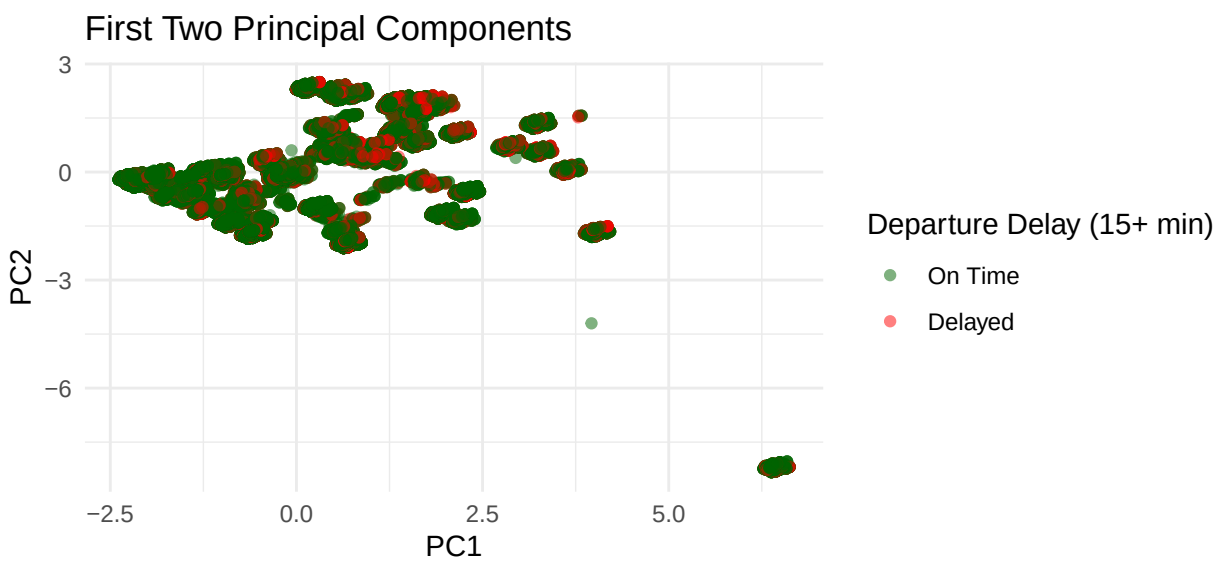

Figure 5: While PCA does create distinct clusters, they do not seem to differentiate delayed status.

3 Supervised Analysis

3.1 Feature Engineering

All predictive modelling methods used in this analysis had the same collection of covariates and the same response variable. The response variable was `dep_delay15`, a binary variable indicating whether a flight was delayed by more than 15 minutes. The EDA suggested that delay risk depends on a mix of stable historical tendencies and day-specific operating conditions, so we engineered features capturing time-of-day effects, carrier and route history, route distance, seasonality, and same-day disruption. These features are defined in Table 3 and were used as covariates in all modelling methods. Several visible-window features were included to capture different aspects of same-day disruption, including overall delay conditions earlier in the day, recent delay patterns, severe disruption events, and carrier-specific disruption. Historical delay-rate features for carriers, destinations, and routes were smoothed toward the overall training delay rate to reduce noise in groups with relatively few observations.

Missing data was handled differently depending on whether it involved the outcome or a predictor. The 950 observations with missing `dep_del15` aligned exactly with likely cancelled flights, so these rows were excluded rather than imputed. For engineered historical delay-rate features, unseen carriers, destinations, or routes in the 2025 data were assigned the global training delay rate as a fallback, since no group-specific history was available. For same-day visible-window features, missing values were handled according to the meaning of the variable: delay rates or average probabilities were replaced with the global training delay rate as a conservative baseline, while counts, maxima, and binary indicators of observed disruption were replaced with 0 to indicate that no such event had yet been observed. In addition, factor levels were explicitly aligned between the training and prediction data before building model matrices so that all methods used a consistent predictor structure.

Table 3: The following features were engineered based on the EDA results and uniformly plugged into all machine learning methods

Feature	Definition
`sched_dep_hour`, `sched_arr_hour`	scheduled departure hour, scheduled arrival hour
`is_late_arrival`	if the arrival time is between 9pm and 1am
`crs_elapsed_time`	scheduled flight duration in minutes
`carrier_size`	categorizing carriers by number of incoming flights at PIT in 2023-2025 with "large" for over 5000, "medium" for between 1000-5000 flights, and "small" otherwise
`dest_region`	groups destination into "northeast", "south", "midwest", "southwest", "west" and "other"
`is_hub_dest`	if an airport is a major hub airport that tends to experience most congestion
`distance`, `distance_bucket`	the route distance in miles as both a continuous variable and a three-level factor (">600", "301-600", <=300")
`month`, `day_of_week`	month as a number, day of the week
`peak_season`	a binary variable that is true for June, July, August and December, which have high delay rates
`is_morning`, `is_evening`	binary variables for if it is a morning departure (i.e. early than 8am) or if it is an evening departure (i.e. after 6pm), respectively
`carrier_delay_rate_smooth`	the smoothed delay rates for carriers over the training set
`dest_delay_rate_smooth`	the smoothed delay rates for destinatinos over the training set
`route_delay_rate_smooth`	the smoothed delay rates for routes over the training set
`prior_same_day_delay_rate`	the fraction of earlier flights on the same day that were delayed
`before_today`	the number of flights before the current observation on that date
`visible_delay_rate`	the fraction of earlier visible departures on that date that were delayed at least 15 minutes

Feature	Definition
`visible_count`	the number of earlier visible departures on that date before the current flight
`visible_max_delay`	the largest observed departure delay among earlier visible departures on that date
`visible_n_delayed`	the number of earlier visible departures on that date that were delayed at least 15 minutes
`visible_recent_rate`	the mean delayed indicator among the most recent visible departures, used to capture short-term disruption
`visible_severe_delay`	indicator for whether the visible window that day contains at least one severe departure delay (> 60 minutes)
`visible_carrier_delay_rate`	the fraction of earlier visible departures for the same carrier on that date that were delayed
`visible_carrier_n`	the number of earlier visible departures for the same carrier on that date
`flights_same_hour`	the number of flights scheduled in the same departure hour
`high_disruption_calendar_day`	indicator for calendar periods whose weekly historical delay rate is above the 90th percentile

3.2 Machine Learning Techniques

This analysis used a variety of machine learning methods with different variations. As previously mentioned, all methods used the same set of covariates and the same response variable, so differences in performance are attributed to the choice of method rather than the predictors themselves. Since this is a binary classification problem, we compared several classification methods with different strengths in order to identify the best-performing model based on accuracy and AUC. Logistic regression was fit first as a simple and interpretable baseline, allowing us to assess whether the engineered variables contributed meaningful predictive signal rather than mostly adding noise. Logistic regression with LASSO regularization was then fit to reduce instability from correlated covariates. We fit XGBoost with five-fold cross-validation over a grid of tuning parameters and selected the combination that maximized cross-validated AUC. In particular, we tuned `max_depth` to control tree complexity, eta to control the learning rate, subsample and `colsample_bytree` to add row- and column-level randomness, `min_child_weight` to prevent splits on very small noisy groups, and γ to require a minimum improvement before a split was made. The number of boosting rounds was then chosen by early stopping. Because XGBoost can overfit when many engineered variables are correlated, these parameters were tuned on held-out folds to capture nonlinear structure and interactions while controlling variance. We fit a random forest with 300 trees to reduce variance and allow for nonlinear effects and interactions without specifying them manually. We also fit a balanced random forest with the same number of trees to account for the imbalanced base rate. Finally, we created two ensemble predictors, a simple average and a weighted average of the predicted probabilities from the strongest methods. The weighted ensemble placed more weight on the higher-performing models, specifically: $Ensemble_{weighted} = 0.55 \times xgboost + 0.20 \times rf + 0.15 \times lasso + 0.10 \times log_reg)$. Since the balanced random forest did not improve performance relative to the regular random forest, it was not included in the weighted ensemble.

3.3 Testing and Validation

First, the 2023 to 2024 data was divided into a training and validation split with an 80-20 split. The training portion was used to fit the models, and the held-out validation portion was used to compare methods during development. After fitting and tuning on those data, final AUC and accuracy were evaluated on the visible subset of the 2025 flights, which served as a proxy for out-of-sample performance before generating predictions for the guess set. As shown by Table 4, the accuracy rates across the final models are all comparable at around 88%, so AUC was used as the main distinguishing metric. Since the weighted ensemble achieved the strongest visible-set AUC, this analysis selected it as the final model for prediction.

Table 4: The methods are ordered by descending AUC since their accuracy rates are comparable. The ensemble techniques have the strongest AUC values.

Modelling Method	AUC_{2025}	$Accuracy_{2025}$
Ensemble (Weighted Average)	0.6947487	0.8861746
Ensemble (Average)	0.6913612	0.8862799
Log Reg (no regularization)	0.6902226	0.8858587
XGBoost (tuned)	0.6896899	0.8861746
Logistic Regression (w/ LASSO)	0.6839234	0.8860693
Random Forest	0.6757789	0.8864905
Balanced Random Forest	0.6075110	0.8862799

Because XGBoost was one of the strongest nonlinear single-model methods, we examined its feature importance to understand which engineered variables contributed most to prediction. Based on Figure 6, the strongest features were `visible_carrier_delay_rate`, `sched_dep_hour`, `sched_arr_hour`, and `route_delay_rate_smooth`. This suggests that flights are easier to classify when the historical carrier and route patterns align with disruption that is already visible earlier in the day.

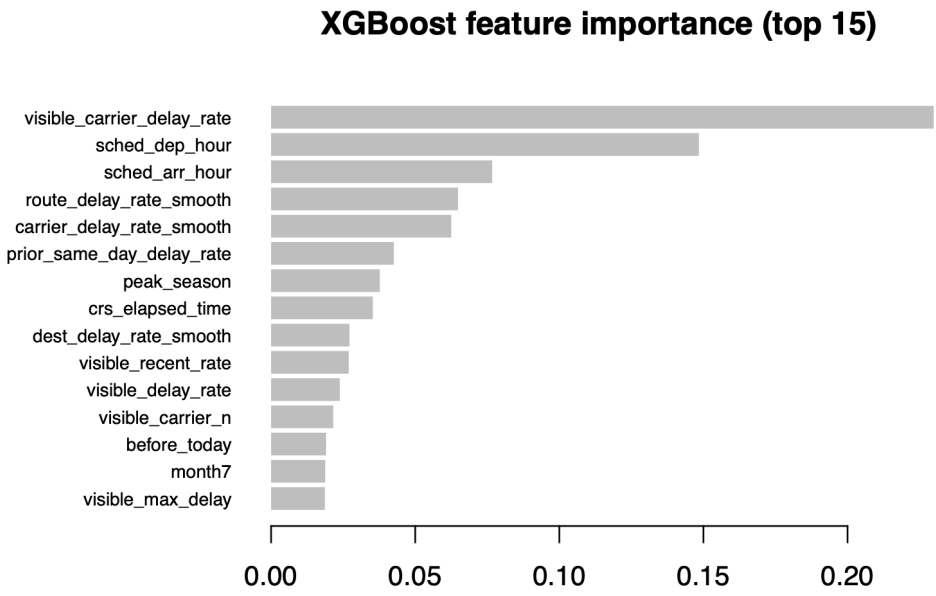

Figure 6: The scheduled departure time, scheduled arrival time, and metrics about related delay rates have the strongest feature importance in the tuned XGBoost model.

4 Analysis of Results

4.1 What kinds of flights do you do well or poorly on?

The models tend to do best on flights whose delay risk is already well-established by the visible-window and historical features. In particular, flights for carriers that have been running late earlier in the day, flights scheduled later in the day, and flights on historically delay-prone routes are easier to classify because several predictors point in the same direction. This is consistent with the XGBoost feature importance in Figure 6.

By contrast, the models do worse on flights whose risk is more ambiguous. This includes flights earlier in the day, when there is less same-day information available, as well as flights on carriers or routes that

do not have strong historical delay patterns. These cases are harder because the predictors do not clearly separate on-time and delayed flights, so the models rely more on weaker background signals. More broadly, the closeness of the top AUC values in Table 4 suggests that the main limitation is not the modelling method itself, but rather the amount of predictive information available before departure.

4.2 Predicting Minutes Delayed

If this transitioned into a regression problem, where we made continuous predictions about the minutes delayed, we would shift from the AUC loss function to mean absolute error as the primary loss function, since it is suitable for regression and is more robust to outliers than mean squared error when delay times are strongly right-skewed. In addition, we would use regression machine-learning methods with varying complexity and again use an ensemble predictor with both a simple average and a weighted average. To start, we would use linear regression with and without LASSO as a simple baseline, then transition to more complex methods such as XGBoost and random forest.

4.3 Trade-Off of a Fixed Cost

Assume that we cared about a trade-off of a fixed cost C for missing a flight and an hourly cost r of waiting for a delayed flight. We would again transition to a regression problem, where we made continuous predictions about minutes delayed, and use the regression methods described above. However, instead of mean absolute error, we would use the following loss function:

$$\begin{cases} r \times (y - de\hat{l}ay)^2 \ if \ y > \hat{y} \\ \\ C \ if \ y < \hat{y} \\ \\ 0 \ else \end{cases}$$

We shift to squared error for underpredicting delays because we no longer want a loss function that is robust to outliers. In this setting, waiting for a delayed flight has loss proportional to the length of the delay, so large underpredictions should be penalized more heavily. In addition, the loss is asymmetric because there is a proportional loss for underpredicting a true delay and a fixed-rate loss for predicting a delay when the flight is actually on time or early.